

İstanbul Topkapı Üniversitesi

FET302 -Derin Öğrenme 2025-2026 Güz Dönemi



Proje Başlığı

CNN Tabanlı Türk İşaret Dili Tanıma

DeepSign

Ekip Üyeleri

1. Emrullah Yıldız – 23040301101 – emrullahyildiz1@stu.topkapi.edu.tr
2. Hüseyin Kaba – 23040301086 – huseyinkaba@stu.topkapi.edu.tr
3. Ali Zeynel Sönmez – 23040301053 – alizeynelsonmez@stu.topkapi.edu.tr
4. Berkay Aras – 23040301055 – berkayaras@stu.topkapi.edu.tr

Proje Bağlantısı

<https://github.com/Byte52/DeepSign-Proje>

Problem Tanımı ve Motivasyonumuz

Türk İşaret Dili'nde (TİD) kullanılan el hareketlerinin doğru şekilde yorumlanması, işitme engelli bireylerle toplum arasındaki iletişimi kolaylaştırmak için kritik öneme sahiptir. Ancak işaret dili bilmeyen bireylerin sayısı oldukça fazladır ve iletişim çoğu zaman sadece yazı ile sınırlı kalmaktadır. Bu nedenle el işaretlerini otomatik olarak tanıyan bir sistem hem bireyler hem de dijital platformlar için önemli bir ihtiyaçtır. Projedeki motivasyonumuz sosyal açıdan engelli bireyler için iletişim bariyerini kırmak teknik açıdan ise CNN mimarilerinin görüntü sınıflandırmadaki davranışlarını anlayıp analiz etmektir.

Projede hedefimiz farklı CNN modellerinin Türk işaret dilindeki el işaretlerini ne kadar doğru sınıflandırabileceğini görmek ve aralarında performans karşılaştırması yapmaktır. Veri setindeki her bir el işareti görüntüsü, önceden tanımlanmış 29 adet TİD sınıfından yalnızca birine atanacaktır.

Hedef Değişkenler ve Başarı Kriterleri

Projede kullanılacak veri setinde hedef değişken el hareketini temsil eden sınıftır ve türü kategoriktir. Veri seti 29 sınıftan oluşmakta olup sınıflar harf etiketi şeklindedir.

Projede kullanılacak temel değerlendirme metrikleri Accuracy, Precision, Recall, F1-score, Confusion matrix olup bu metrikler her bir model için hesaplanacaktır. Nicel hedeflerimiz şunlardır:

$$\text{Accuracy} \geq 0.85 \text{ F1-score} \geq 0.80$$

Proje Yönetimi

Proje Zaman Çizelgesi

- 1. Hafta (20- 27 Ekim):** Proje kapsamının belirlenmesi (TİD Alfabesi veya Kelimeler), literatürdeki mevcut TİD çalışmalarının incelenmesi.
- 2. Hafta (27 Ekim- 3 Kasım):** Veri Seti Oluşturma: Hazır TİD veri setlerinin (BosphorusSign vb.) incelenmesi veya grup üyeleri tarafından kamera karşısında özgün TİD görüntülerinin kaydedilmesi.
- 3-4. Hafta (3- 17 Kasım):** Veri Ön İşleme: Görüntülerin kırılması (ROI), el tespiti ve görüntülerin gri tona çevrilerek etiketlenmesi.
- 5. Hafta (17- 24 Kasım):** GitHub reposunun ("DeepSign-Proje") oluşturulması, ekip erişimlerinin verilmesi ve temel CNN mimarisinin TİD sınıflarına göre kurgulanması.
- 6. Hafta (24 Kasım- 1 Aralık):** Modelin eğitilmesi, Türk İşaret Dili'ndeki benzer el hareketlerinin (Örn: 'O' ve '0' veya benzer harfler) ayırt edilmesi üzerine model iyileştirmeleri.
- 7. Hafta (1- 8 Aralık):** Hiperparametre optimizasyonu ve modelin test edilmesi.
- 8. Hafta (8- 15 Aralık):** Proje raporunun yazılması, TİD performansının analiz edilmesi ve sunumun hazırlanması.

Grup Üyeleri Roller

Berkay Aras: LeNet-5 mimarisini PyTorch'ta sıfırdan tasarlayacak ve uygulayacaktır. Modelin genelleme yeteneğini incelemek için farklı Dropout Oranları (0.0, 0.25, 0.50) üzerinde deneyler yapacaktır. En yüksek F1-Score sonucunu veren Dropout oranı, LeNet-5 için optimize edilmiş hiperparametre olarak belirlenecektir. Final raporu için VGG16 ve Resnet50 derin öğrenme modelini geliştirmeyi planlamaktadır.

Hüseyin Kaba: AlexNet'i PyTorch'ta uygulayacaktır. Modelin evrimsel gövdesi dondurulup, yeni sınıflandırıcı başlığı TİD verisi ile eğitilecektir. Modelin genelleme sınırını belirlemek amacıyla farklı öğrenme hızı değerleri test edilecek ve bu denemelerle birlikte epoch sayısı optimizasyonu yapılacaktır. Final raporu için MobileNetV2 ve ShuffleNet derin öğrenme modelini geliştirmeyi planlamaktadır.

Emrullah Yıldız: 3 evrim katmanı içeren sıkı bir custom CNN modeli geliştirecektir. Modelin eğitim verimliliğini ve bellek kullanımını incelemek amacıyla farklı batch size değerlerini deneyerek genelleme yeteneğine olan etkisini analiz edecektir. Final raporu için ResNet-18 ve DenseNet121 derin öğrenme modelini geliştirmeyi planlamaktadır.

Ali Zeynel Sönmez: Emrullah Yıldız'ın modelinden farklı bir kernel size veya pooling boyutu ile farklılaştırılmış ikinci bir custom CNN modeli geliştirecektir. Üye 4, modelin optimizasyon dinamiklerini incelemek amacıyla, aynı eğitim koşulları altında adam ve SGD optimizier'larının modelin final performansı ve eğitim eğrileri üzerindeki etkilerini karşılaştıracaktır. Final raporu için EfficientNet ve InceptionV3 derin öğrenme modelini geliştirmeyi planlamaktadır.

Temel Kaynaklar:

Sandler et al. (2018) MobileNetV2 (CVPR): Bu çalışma, mobil cihazlarda gerçek zamanlı görüntü sınıflandırma için önerilmiş hafif bir CNN mimarisi olan MobileNetV2'yi sunmaktadır. Yöntem, düşük parametre sayısı sayesinde hızlı eğitim süresi sunmakta ve işaret dili dahil birçok el işareti veri setinde %90 üzerinde doğruluk sağladığı rapor edilmiştir.

He et al. (2016) ResNet (CVPR): Bu çalışma, derin ağların öğrenmesini kolaylaştıran Residual bağlantı (Skip Connection) yapısıyla tanınan ResNet mimarisini tanıtmaktadır. ResNet, derin mimarisi sayesinde karmaşık örüntüleri etkin şekilde öğrenmekte ve işaret dili sınıflandırma çalışmalarında %95 ve üzeri doğruluk oranlarına ulaştığı bildirilmiştir.

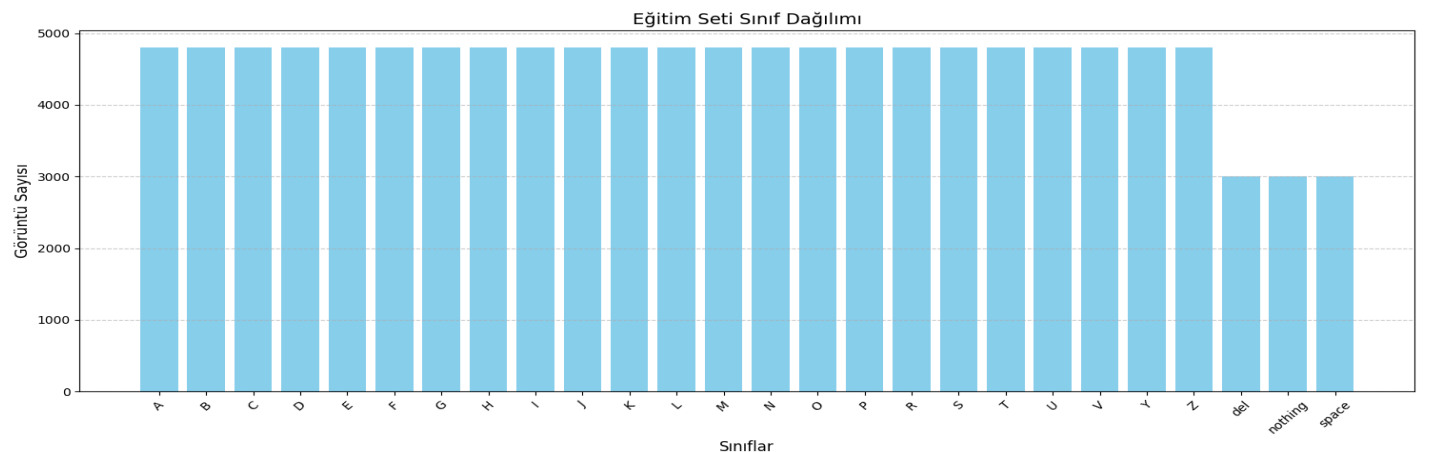
Tan & Le (2019) EfficientNet (ICML): EfficientNet, derinlik, genişlik ve çözünürlük ölçeklendirmesini optimize eden modern bir CNN ailesidir. Bu yöntem, Bileşik Ölçekleme kullanarak yüksek verimlilik sunmakta ve işaret dili tanıma araştırmalarında %94- %98 doğruluk aralığında başarılı sonuçlar vermektedir.

Kaynak	Kapsam (Yöntem Odak Noktası)	Metrikler ve Raporlanan Sonuçlar
MobileNetV2	Düşük hesaplama maliyeti.	%90- %95 Doğruluk.
ResNet	Derinliği artırma	%95+ Doğruluk
EfficientNet	Bileşik ölçekleme ile verimlilik	%94- %98 Doğruluk

Genel olarak, çalışmaların çoğunda accuracy ve F1-score metrikleri kullanılmıştır. MobileNetV2 gibi hafif mimariler düşük hesaplama maliyeti sunarken, ResNet ve EfficientNet gibi derin mimariler daha yüksek doğruluk sağlamıştır. MobileNetV2 ve EfficientNet, mobil uyumluluğu öne çıkarırken, ResNet daha saf bir akademik derinlik sunmaktadır.

Veri Açıklaması ve Yönetimi

Projede kullanılacak temel veri kümesi TR Sign Language Dataset (Kaggle)'tir. Bu veri kümesi, Türk İşaret Dili (TİD) alfabesine ait 29 farklı sınıfı ve toplamda yaklaşık 167.186 görüntü içermektedir. Görüntüler, farklı ışıklandırma ve arka plan koşullarında çekildiği için, eğitilen modelin genelleme yeteneğini test etmek için gerçekçi bir zemin sunuyor. Veri seti, 29 sınıf (harf) içermektedir ve her sınıfta yaklaşık 4800 görsel (del , nothing ve space sınıfları hariç) bulunmaktadır. Toplamda yaklaşık 167.186 görüntü (data point) içerir. Görsellerin boyutu 200×200 RGB piksel şeklindedir. Görseller 128*128*3 RGB boyutunda yeniden boyutlandırılacaktır. Bu boyutlandırma modele göre değişiklik gösterebilir. Ayrıca veri kümesi genel kullanıma açık bir kaynaktan (Kaggle) bulunmaktadır ve kişisel yüz veya kimlik bilgilerini içermediği için doğrudan bir gizlilik veya kimlik ifşa etme riski taşımamaktadır. Modelin sadece el işaretlerini sınıflandırmaya odaklanması, etik riski düşürmektedir. Veri kümesi orijinal olarak train ve test klasörlerinden oluşmaktadır. Validation seti ise train içinden alınarak kullanılmıştır. Bu üç set, hiperparametre optimizasyonu ve model kararlılığı kontrolü için kullanılacaktır.



Base Mimari Modeller ve Kullanılacak Yöntemler

- Tüm görüntüler 128*128 olacak şekilde yeniden boyutlandırılacaktır.
- Görüntülerin piksel değerleri [0, 1] aralığına normalize edilecektir.
- Veri kümesi; train, validation ve test setleri olmak üzere üç ayrı küme halinde ayrılacaktır.
- Aşırı öğrenmeyi engellemek ve genelleme yeteneğini artırmak amacıyla rastgele döndürme, yatay çevirme ve parlaklık değişimi gibi teknikler uygulanacaktır.
- Tüm sınıflandırma modelleri için cross-entropy loss kullanılacaktır.
- Genel olarak Adam optimizer' ı kullanılacaktır.
- Validation kaybının 5 epoch boyunca iyileşme göstermediği durumlarda erken durdurma mekanizması uygulanacaktır.

Her Grup Üyesinin Geliştirdiği Base Model ve Detayları

1. Berkay Aras (LeNet-5)

LeNet-5 modeli PyTorch'ta sıfırdan uygulanmıştır. Model 2 evrişim katmanı, 2 max pooling katmanı ve yoğun katmandan oluşmaktadır. Kernel size orijinal tasarıma uygun olarak 5*5 olarak belirlenmiştir. Görüntüler 128*128 olarak yeniden boyutlandırılmıştır. Eğitim aşaması adam optimizieri ve cross-entropy loss kullanılarak gerçekleştirilmiştir. Orijinal eğitim verisinin %80'i train, %20'si validation olarak kullanılmıştır. Test kümesi olarak ise ayrı bir set bulunmaktadır.

Hiperparametre Optimizasyonu ve Yapılan Deney

Modelin genelleme yeteneğini ve ezberleme eğilimini incelemek amacıyla Dropout oranları üzerinden deney yapılmıştır. Modelde Dropout değeri 0.0/0.25/0.50 olmak üzere üç farklı oranla ayrı ayrı eğitilmiştir. Yapılan deneyler sonucunda en yüksek performansı "0.0" değeri sağlamıştır.

Dropout Rate	Accuracy	F1-Score	Best Epoch
0.00	0.8727	0.8794	19
0.25	0.7977	0.8080	20
0.50	0.6467	0.6663	19

2. Emrullah Yıldız (Custom CNN)

Eğitim öncesinde tüm görüntüler donanım optimizasyonu için 64x64 piksel boyutuna indirgenmiş; gradyan kararlılığını sağlamak amacıyla piksel değerleri [-1, 1] aralığına normalize edilmiştir. Geliştirilen mimari, öznetelik çıkarımı yapan üç ardışık evrişim bloğundan oluşmaktadır. Kanal derinliği bloklar boyunca sırasıyla 32, 64 ve 128'e çıkarılmış, ardından veriler düzleştirilerek 512 nöronlu tam bağlantılı katman ile sınıflandırma gerçekleştirilmiştir. Validation seti veri setinde olmadığı için test veri setinin %100'ü alınıp kullanılmıştır.

Hiperparametre Optimizasyonu ve Yapılan Deney

Modelin performansını etkileyen Batch Size üzerinde 32, 64 ve 128 değerleri ile karşılaştırmalı deneyler yapılmıştır. Validasyon seti üzerindeki F1-Score ve yakınsama hızı analizleri sonucunda, en kararlı ve başarılı performansın batch size 64 ile elde edildiği tespit edilmiştir.

Best Batch Size: 64 (F1-Score: 0.9499)

3. Hüseyin Kaba (AlexNet)

Veri ön işleme aşamasında tüm görüntüler 128*128 boyutuna getirilmiş, normalize edilmiş ve genelleme başarısını artırmak için döndürme, yatay çevirme ve zoom gibi veri artırma yöntemleri uygulanmıştır. Eğitim sürecinde CrossEntropyLoss ve Adam optimizer kullanılmış, yapılan denemeler sonucunda 0.0001 öğrenme oranı ve yaklaşık 25 epoch'luk yapı en dengeli sonucu vermiştir. Ayrıca %50 dropout ve 1e-5 weight decay ile aşırı öğrenme azaltılmıştır.

Hiperparametre Optimizasyonu ve Yapılan Deney

Denemelerde farklı öğrenme oranları (0.001, 0.0005 ve 0.0001) test edilmiş ve en iyi sonucu 0.0001 learning rate vermiştir. Eğitim süresi için yapılan karşılaştırmalarda 25 epoch en uygun değer olarak belirlenmiştir. Batch size olarak 128, dropout oranı %50 belirlenmiştir.

Accuracy: 0.9327558791530672
Precision: 0.935405354454937
Recall: 0.9327558791530672
F1: 0.9330566871749096

4. Ali Zeynel Sönmez (Custom CNN)

Model 128×128 boyutuna ölçeklenmiş görüntüleri giriş olarak almaktadır. İlk convolution katmanı 16 filtre, ikinci katman 32 filtre kullanmaktadır. Her katmandan sonra ReLU ve MaxPool uygulanarak hem özellik çıkarımı hem de boyut azaltma yapılmıştır. Sonrasında veriler flatten edilerek 128 nöronlu bir FC katmanına, oradan da 29 sınıflı çıkış katmanına aktarılmaktadır.

Hiperparametre Optimizasyonu ve Yapılan Deney

Farklı batch size değerleri 32, 64 ve 128 test edilmiş ve en iyi performansı batch size 64 vermiştir. Model toplam 3 epoch boyunca eğitilmiştir.

```
accuracy      0.80      47796
macro avg     0.82      0.81      0.81      47796
weighted avg  0.81      0.80      0.80      47796
```

Hiperparametre optimizasyonu, her grup üyesinin bireysel olarak farklı bir parametre üzerinde karşılaştırmalı denemeler yapmasıyla gerçekleşmiştir. Optimize edilmiş hiperparametre ilgili deneme setinde validation üzerinde en iyi F1 skoru veren değer olarak seçilmiştir.

Grup Üyesi	Uygulanan Base Model	Hiperparametre	Accuracy	F1-Score	Optimize Edilmiş Değer
1. Berkay Aras	LeNet-5	Dropout Oranı (0.00-0.25-0.50)	0.8727	0.8794	Dropout: 0.0
2. Emrullah Yıldız	Custom CNN	Batch size (32, 64, 128)	0.95	0.9499	Batch size: 64
3. Hüseyin Kaba	AlexNet	Öğrenme Oranı (0.001, 0.0005 ve 0.0001)	0.9327	0.9330	LR: 0.0001
4. Ali Z. Sönmez	Custom CNN	Batch size (32, 64, 128)	0.80	0.81	Batch size: 64

Base Modellerin Sonuç Analizi

Ali Z. Sönmez'in custom CNN modeli hariç diğer modeller nicel hedeflerimiz olan Accuracy $\geq$ 0.85 ve F1-Score $\geq$ 0.80 değerlerini başarıyla aşmıştır. Dört farklı model arasından en iyi F1 skoru veren model 0.9499 skoru ile Emrullah Yıldız'ın geliştirdiği custom CNN modeli olmuştur.

Emrullah ve Ali'nin Custom CNN modellerinde batch size = 64 değerinin tercih edilmesi, Batch Size'in TİD verisi üzerindeki eğitim kararlılığına olan pozitif etkisini göstermiştir. Ancak Ali Z. Sönmez'in custom CNN modelinin nispeten düşük performansı, bu modelde kullanılan farklı çekirdek boyutu veya katman tasarımının, diğer basit modellere göre TİD için uygun olmadığını göstermektedir.

Model	Accuracy	Precision	Recall	F1-Score
AlexNet	0.9327	0.9354	0.9327	0.9330
LeNet5	0.8727	0.8223	0.8922	0.8794
CustomCNN1	0.9532	0.9359	0.9487	0.9499
CustomCNN2	0.8232	0.8123	0.8245	0.8145

Gelişmiş Derin Öğrenme Modelleri ve Deneysel Sonuçlar

Bu bölümde vize aşamasında kullanılan temel CNN modellerine ek olarak, final aşamasında denenen gelişmiş transfer learning modelleri açıklanmaktadır. Bu kapsamda sekiz farklı derin mimari üzerinde çalışılmıştır.

Çalışılan Modeller:

- VGG16
- Resnet-50
- Resnet-18
- DenseNet-121
- MobileNetV2
- ShuffleNetV2
- EfficientNet-B0
- InceptionV3

Tüm bu modellerde ortak amaç, Türk İşaret Dili el işaretlerini 26 sınıfta mümkün olan en yüksek doğrulukla sınıflandırmak ve aynı veri işleme hattı ile adil bir karşılaştırma yapmaktır.

Ortak Eğitim ve Değerlendirme Çerçevesi

Gelişmiş modelleri karşılaştırılabilir kılmak için bütün deneylerde aşağıdaki ortak yapı izlenmiştir.

Veri Seti ve Ön İşleme Stratejisi

- Tüm görüntüler 128*128 boyutuna yeniden ölçeklendirilmiş, RGB formatında modele verilmiştir.
- Piksel değerleri, tüm modeller için ortak olacak şekilde mean= [0.5,0.5,0.5], std= [0.5,0.5,0.5] parametreleriyle normalize edilmiştir.
- Eğitim aşamasında RandomHorizontalFlip, RandomRotation gibi basit artırma işlemleri kullanılarak modelin farklı el pozisyonlarına ve küçük dönüşlere karşı dayanıklılığı artırılmıştır.
- Veri seti, proje protokolüne uygun olarak yaklaşık %80 eğitim – %20 doğrulama olacak şekilde ayrılmış, Kaggle veri setindeki test klasörü tüm üyeler tarafından ortak test seti olarak kullanılmıştır.

Transfer Learning Stratejisi

Tüm gelişmiş modeller ImageNet üzerinde pretrained ağırlıklarla başlatılmıştır. Her modelin son katmanı, 26 çıktıya yeni bir lineer katman ile değiştirilmiştir.

İki aşamalı strateji uygulanmıştır:

1. **Freeze Aşaması:** Sadece en son sınıflandırma katmanı eğitilmiş, diğer katmanların ağırlıkları dondurulmuştur. Bu aşama boyunca farklı öğrenme oranı (LR), batch size ve optimizasyon kombinasyonları denenmiş, her model için en iyi doğrulama başarımı veren konfigürasyon seçilmiştir.
2. **Fine-Tuning Aşaması:** En iyi konfigürasyon belirlendikten sonra, modelin üst katmanları kısmen açılarak tüm ağırlık düşük öğrenme oranı ile fine tuning yapılmıştır.

Eğitim Parametreleri

Kayıp fonksiyonu olarak Cross-Entropy Loss kullanılmıştır. Optimizasyon algoritmalarında ise çoğu deneyde Adam algoritması kullanılmış olup bazı denemelerde karşılaştırma amaçlı SGD + momentum kullanılmıştır. Öğrenme oranı aralıkları $1 \cdot 10^{-3}$, $5 \cdot 10^{-4}$, $3 \cdot 10^{-4}$ gibi değerler taranmış, her model için en iyi değer rapor edilmiştir.

Batch size 64 ve 128 olarak denenmiş en iyi sonucu veren batch size ilgili model altında belirtilmiştir. Epoch sayısı ise freeze aşaması için genellikle 10 epoch fine tuning aşaması için ise ek 5 epoch eğitim yapılmıştır. Overfitting gözlenmemesi için eğitim ve doğrulama kayıpları takip edilmiştir.

Değerlendirme Metrikleri

Tüm modeller ortak test seti üzerinde aşağıdaki makro metriklerle değerlendirilmiştir.

- Accuracy
- Macro Precision
- Macro Recall
- Macro F1-Score

Ayrıca her model için sınıf bazlı F1 skorları hesaplanmış, hangi modelin hangi sınıfta lider olduğu analiz edilmiştir. Sonuçlar daha sonra tek bir tabloda ve çeşitli grafiklerle karşılaştırılmıştır.

Her Ekip Üyesinin Geliştirmiş Olduğu Gelişmiş Modeller ve Sonuçları

1) VGG16

VGG16, daha derin ama yapısal olarak sade klasik bir CNN mimarisidir ve bu nedenle hem referans hem de karşılaştırma modeli olarak kullanılmıştır. Model, ImageNet ağırlıkları ile başlatılmış, ilk aşamada son sınıflandırma katmanı eğitime açılarak freeze training yapılmıştır. Bu aşamada modelin, veri setindeki temel desenleri yakalamakta zorlanmadığı, ancak daha yüksek doğruluk seviyelerine ulaşmak için ince ayara ihtiyaç duyduğu görülmüştür. Bu yüzden modele fine tuning uygulanmış olup, özellikle üst katman blokları eğitime dahil edilerek veri setinin istatistiklerine daha spesifik hale gelmesi sağlanmıştır.

Hiperparametre Optimizasyonu ve Deneyler

Hiperparametre optimizasyonu kapsamında, öğrenme oranı 0.001, 0.0005 ve 0.0003 değerleri ile denenmiş; batch boyutu 128 olarak seçilmiş; optimizier tarafında Adam ve SGD karşılaştırılmıştır. Deneyler sonucunda, öğrenme oranı 0.0003, batch size 128 ve Adam optimizier kombinasyonu en yüksek ve en kararlı test performansını sağlamıştır. Bu konfigürasyon ile VGG16, test setinde yaklaşık %99,39 doğruluk ve %99,42–%99,43 civarında macro precision, recall ve F1-score değerlerine ulaşmıştır.

```
===== Tuning Sonuçları =====
({'lr': 0.001, 'batch': 64, 'opt': 'adam', 'dropout': 0.5, 'freeze': True}, 0.9254606365159129)
({'lr': 0.0005, 'batch': 64, 'opt': 'adam', 'dropout': 0.5, 'freeze': True}, 0.9696398659966499)
({'lr': 0.0003, 'batch': 128, 'opt': 'adam', 'dropout': 0.5, 'freeze': True}, 0.9859715242881072)
({'lr': 0.0005, 'batch': 64, 'opt': 'sgd', 'dropout': 0.5, 'freeze': True}, 0.9461892797319933)
({'lr': 0.0005, 'batch': 64, 'opt': 'adam', 'dropout': 0.3, 'freeze': True}, 0.9676716917922948)
```

Bu konfigürasyon ile VGG16, test setinde yaklaşık 0.9939 doğruluk ve 0.9942-0.9943 civarında macro precision, recall ve F1-score değerlerine ulaşmıştır.

```
===== VGG16 TEST SONUÇLARI =====
Accuracy : 0.99391
Precision: 0.99431
Recall   : 0.99424
F1-score : 0.99424
```

Her ne kadar VGG16, DenseNet veya MobileNet kadar parametrik verimli olmasa da, yüksek doğruluk seviyeleriyle güçlü bir referans model sunmuş; çalışmada kullanılan diğer gelişmiş mimarilerin performans kazanımlarını değerlendirmek için kıyaslama noktası işlevi görmüştür.

2) ResNet-50

Model ImageNet üzerinde önceden eğitilmiş ağırlıklarla başlatılmış, ilk aşamada sadece son tam bağlantılı katman eğitilerek freeze training yapılmıştır. Bu aşama, modelin veri setine nasıl tepki verdiğini hızlı bir şekilde görmek için kullanılmıştır. Ardından, ResNet-50 modeli için fine tuning süreci uygulanmış ve son bloklar eğitime dahil edilmiştir.

Hiperparametre Optimizasyonu ve Deneyler

Hiperparametre optimizasyonunda, öğrenme oranı 0.001, 0.0005 ve 0.0003 değerleri ile test edilmiş; batch boyutu 64 olarak belirlenmiş; optimizier olarak yine Adam ve SGD kıyaslanmıştır.

```
===== ResNet50 Tuning Sonuçları =====
({'lr': 0.001, 'batch': 64, 'opt': 'adam'}, 0.8891122278056951)
({'lr': 0.0005, 'batch': 64, 'opt': 'adam'}, 0.8909128978224455)
({'lr': 0.0003, 'batch': 128, 'opt': 'adam'}, 0.8736599664991624)
({'lr': 0.0005, 'batch': 64, 'opt': 'sgd'}, 0.8410804020100503)
```

Deney sonuçları, öğrenme oranı 0.0005, batch size 64 ve Adam optimizier kombinasyonunun en iyi doğrulama ve test performansını verdiğini göstermiştir. Bu yapılandırma ile model test setinde yaklaşık %99,55 doğruluk ve %99,5 civarında macro precision, recall ve F1-score değerlerine ulaşmıştır.

```
===== RESNET50 TEST SONUÇLARI =====
Accuracy : 0.99548
Precision: 0.99523
Recall   : 0.99515
F1-score : 0.99515
```

3) ResNet-18

ResNet-18 modeli, derin ağlarda görülen gradyan kaybolması problemini azaltan hafif ama güçlü bir mimari olduğundan, çalışmada gelişmiş modellerden biri olarak tercih edilmiştir. Model, ImageNet ağırlıklarıyla başlatılmış ve ilk aşamada sadece son tam bağlantılı katman eğitime açılarak freeze training uygulanmıştır. Bu aşamada 10 epoch boyunca yapılan eğitim sonucunda eğitim doğruluğu yaklaşık %53, doğrulama doğruluğu ise %72 civarında seyretmiştir.

Hiperparametre Optimizasyonu ve Deneyler

```
===== ResNet18 Tuning Sonuçları =====
({'lr': 0.001, 'batch': 64, 'opt': 'adam'}, 0.7001675041876047)
({'lr': 0.0005, 'batch': 64, 'opt': 'adam'}, 0.7246231155778895)
({'lr': 0.0003, 'batch': 128, 'opt': 'adam'}, 0.7086264656616416)
({'lr': 0.0005, 'batch': 64, 'opt': 'sgd'}, 0.7069514237855946)
```

Hiperparametre aramasında öğrenme oranı için 0.001, 0.0005 ve 0.0003 değerleri test edilmiş; batch boyutu 64 olarak sabit tutulmuş; optimizier olarak ise Adam ve SGD karşılaştırılmıştır. Yapılan deneyler sonucunda 0.0005 öğrenme oranı, batch size 64 ve Adam optimizier kombinasyonu hem doğrulama hem de test seti üzerinde en dengeli ve en yüksek performansı sunmuştur. Bu konfigürasyon ile yapılan fine tuning sonrasında model, test seti üzerinde %99,67 doğruluk ve yaklaşık %99,69 macro precision, recall ve F1-score değerlerine ulaşmıştır.

```
===== RESNET18 TEST SONUÇLARI =====
Accuracy : 0.99669
Precision: 0.99692
Recall   : 0.99689
F1-score : 0.99688
```

Sınıf bazlı F1 skorları incelendiğinde, çoğu sınıf için %99 ve üzeri sonuçlar elde edilmiş, yanlış sınıflandırmaların oldukça nadir ve sınırlı sayıda olduğu gözlemlenmiştir. Bu bulgular, ResNet-18'in hem model boyutu hem de başarı oranı açısından oldukça verimli bir çözüm sunduğunu göstermektedir.

4) DenseNet121

Model, ImageNet üzerinde önceden eğitilmiş ağırlıklarla başlatılmış ve transfer öğrenme yaklaşımıyla Türk İşaret Dili veri setine uyarlanmıştır. İlk aşamada modelin genelleme kabiliyetini hızlıca gözlemleyebilmek için “freeze training” stratejisi kullanılmış, tüm konvolüsyon katmanları dondurularak yalnızca son sınıflandırma katmanı eğitime açılmıştır. Bu aşamada 10 epoch boyunca eğitim yapılmış ve yaklaşık %63 civarında bir eğitim doğruluğu, %80 düzeyinde bir doğrulama doğruluğu elde edilmiştir.

Hiperparametre Optimizasyonu ve Deneyler

```
DenseNet-121 için Final Freeze Eğitim başlıyor: {'lr': 0.0005, 'batch': 64, 'opt': 'adam'}
Epoch 1/10 - Train Loss: 1.6404, Acc: 0.5201 | Val Loss: 0.9715, Acc: 0.7657
Epoch 2/10 - Train Loss: 1.2674, Acc: 0.6098 | Val Loss: 0.8291, Acc: 0.7897
Epoch 3/10 - Train Loss: 1.2241, Acc: 0.6167 | Val Loss: 0.7794, Acc: 0.7923
Epoch 4/10 - Train Loss: 1.2258, Acc: 0.6155 | Val Loss: 0.7589, Acc: 0.7936
Epoch 5/10 - Train Loss: 1.2061, Acc: 0.6229 | Val Loss: 0.7434, Acc: 0.8044
Epoch 6/10 - Train Loss: 1.2030, Acc: 0.6232 | Val Loss: 0.7409, Acc: 0.7979
Epoch 7/10 - Train Loss: 1.1979, Acc: 0.6257 | Val Loss: 0.7011, Acc: 0.8124
Epoch 8/10 - Train Loss: 1.1975, Acc: 0.6242 | Val Loss: 0.7081, Acc: 0.8106
Epoch 9/10 - Train Loss: 1.1883, Acc: 0.6278 | Val Loss: 0.7084, Acc: 0.8074
Epoch 10/10 - Train Loss: 1.1826, Acc: 0.6292 | Val Loss: 0.7107, Acc: 0.8036
```

İkinci aşamada, model kademeli olarak fine tune edilmiştir. Bu adımda gövde katmanları da eğitime dahil edilmiş, ancak öğrenme oranı düşürülerek (özellikle önceden öğrenilmiş özellikleri bozmamak için) daha kontrollü bir güncelleme yapılmıştır. Öğrenme oranı için 0.001, 0.0005 ve 0.0003 değerleri; batch boyutu için 32 ve 64 değerleri, optimizier tarafında ise Adam ve SGD seçenekleri denenmiştir. Yapılan bu hiperparametre taraması sonucunda en iyi performansın öğrenme oranı 0.0005, batch size 64 ve Adam optimizier kombinasyonunda elde edildiği görülmüştür.

```
===== DenseNet121 Tuning Sonuclari =====
({'lr': 0.001, 'batch': 64, 'opt': 'adam'}, 0.7913735343383584)
({'lr': 0.0005, 'batch': 64, 'opt': 'adam'}, 0.7964405360134004)
({'lr': 0.0003, 'batch': 128, 'opt': 'adam'}, 0.7874371859296483)
({'lr': 0.0005, 'batch': 64, 'opt': 'sgd'}, 0.7640284757118928)
```

Bu yapılandırma ile 10 epoch freeze + 5 epoch fine-tune eğitimi sonrasında test seti üzerinde yaklaşık %99,88 doğruluk ve %99,89 macro precision, recall ve F1-score değerlerine ulaşılmıştır.

```
===== DenseNet121 TEST SONUÇLARI =====
Accuracy : 0.99883
Precision: 0.99890
Recall : 0.99889
F1-score : 0.99889
```

5) MobileNetV2

Bu modelde temel strateji, gövdeyi tamamen dondurup sadece son sınıflandırma katmanını eğitmek üzerine kurulmuştur. Yani klasik anlamda tüm katmanların fine-tune edildiği bir yapı yerine, transfer learning + yalnızca son katmanın eğitimi yaklaşımı uygulanmıştır. Bu sayede hem eğitim süresi ciddi şekilde kısaltılmış hem de küçük bir sınıflandırıcı katman ile yüksek başarı elde edilmesi hedeflenmiştir.

Hiperparametre Optimizasyonu ve Deneyler

Hiperparametre optimizasyonu kapsamında, öğrenme oranı değerleri 0.001, 0.0005 ve 0.0003 olarak denenmiş; batch boyutu 64 olarak seçilmiş ve Adam ile SGD optimizier'ları karşılaştırılmıştır. Deneyler sonucunda öğrenme oranı 0.0005, batch size 64 ve SGD optimizier kombinasyonu, en kararlı ve en yüksek performansı sunmuştur. Modelin gövdesi tamamen dondurulmuş olmasına rağmen, bu konfigürasyon ile test seti üzerinde %99,88 doğruluk ve %99,89 macro precision, recall ve F1-score seviyesine ulaşılmıştır.

```
===== Tuning Bitti - Sonuçlar =====
```

```
({'lr': 0.001, 'batch': 64, 'opt': 'adam'}, 98.91122278056952)
({'lr': 0.0005, 'batch': 64, 'opt': 'adam'}, 99.40536013400335)
({'lr': 0.0003, 'batch': 128, 'opt': 'adam'}, 99.08710217755444)
({'lr': 0.0005, 'batch': 64, 'opt': 'sgd'}, 99.49748743718592)
```

```
===== MobileNetV2 TEST SONUÇLARI =====
```

```
Accuracy : 0.99883
Precision: 0.99889
Recall : 0.99889
F1-score : 0.99889
```

6) ShuffleNetV2

ShuffleNetV2 de yine transfer learning ile kullanılmış, gövde ağırlıkları dondurularak sadece son sınıflandırma katmanı eğitime açılmıştır. Böylece, modelin hız avantajını korurken aynı zamanda işaret dili sınıflarına hızlı uyum sağlaması amaçlanmıştır.

Hiperparametre Optimizasyonu ve Deneyler

Hiperparametre aramasında MobileNetV2'ye benzer bir yaklaşım izlenmiş; öğrenme oranı için 0.001, 0.0005 ve 0.0003 değerleri, optimizer olarak Adam ve SGD seçenekleri denenmiş, batch boyutu 64 olarak seçilmiştir. Yapılan deneyler sonucunda 0.0005 öğrenme oranı, batch size 64 ve Adam optimizer kombinasyonu en iyi doğrulama ve test başarımını sağlamıştır.

```
{'lr': 0.001, 'batch': 64, 'opt': 'adam'}, 99.38442211055276)
{'lr': 0.0005, 'batch': 64, 'opt': 'adam'}, 99.5393634840871)
{'lr': 0.0003, 'batch': 128, 'opt': 'adam'}, 99.51005025125627)
{'lr': 0.0005, 'batch': 64, 'opt': 'sgd'}, 39.98743718592965)
ShuffleNetV2 FINAL TEST ACC: 99.81%
```

Bu ayar ile ShuffleNetV2 test setinde yaklaşık %99,8 doğruluk ve macro metriklerde %99,8 seviyesinde precision, recall ve F1-score elde etmiştir.

```
===== ShuffleNetV2 TEST SONUÇLARI =====
Accuracy : 0.99807
Precision: 0.99818
Recall   : 0.99818
F1-score : 0.99818
```

7) EfficientNet-B0

Bu modelde de ImageNet ön-eğitilmiş ağırlıklar kullanılmış, önce freeze training ile sadece son katman eğitilmiş, ardından fine tuning ile ağırlıklar daha geniş bir kısmı eğitime dahil edilmiştir. Amaç, EfficientNet' in dengeli ölçeklendirme yapısının Türk İşaret Dili veri setine ne kadar iyi uyum sağlayacağını gözlemlemek olmuştur.

Hiperparametre Optimizasyonu ve Deneyler

Hiperparametre aramasında, öğrenme oranı olarak 0.001, 0.0005 ve 0.0003; batch boyutu olarak 64 ve 128; optimizer olarak ise Adam ve SGD denenmiştir. Farklı kombinasyonlar test edildikten sonra, öğrenme oranı 0.001, batch size 64 ve Adam optimizer kombinasyonunun, doğrulama doğruluğu ve genel stabilite açısından en iyi sonucu verdiği görülmüştür.

```
===== EfficientNet-B0 Tuning Sonuçları =====
{'config': {'lr': 0.001, 'batch': 64, 'opt': 'adam'}, 'best_val_acc': 0.8461474036850921}
{'config': {'lr': 0.0005, 'batch': 64, 'opt': 'adam'}, 'best_val_acc': 0.834463986599665}
{'config': {'lr': 0.0003, 'batch': 128, 'opt': 'adam'}, 'best_val_acc': 0.8029731993299832}
{'config': {'lr': 0.0005, 'batch': 64, 'opt': 'sgd'}, 'best_val_acc': 0.7293969849246231}
```

Bu konfigürasyon ile yapılan eğitimler sonucunda EfficientNet-B0 modelinin test seti üzerinde yaklaşık %96,96 doğruluk ve %97,13 macro F1-score elde ettiği gözlemlenmiştir.

```
===== EfficientNet-B0 TEST SONUÇLARI =====
Accuracy : 0.96968
Precision: 0.97157
Recall   : 0.97136
F1-score : 0.97132
```

Her ne kadar sonuçlar genel olarak başarılı olsa da DenseNet-121 ve MobileNetV2 gibi modellerin ulaştığı %99+ seviyesinin bir miktar altında kalmıştır.

8) InceptionV3

InceptionV3 ImageNet ağırlıklarıyla başlatılmış, başlangıçta gövde katmanları dondurularak sadece sınıflandırma katmanı eğitilmiştir. İlk denemelerde modelin karmaşık yapısı nedeniyle öğrenme sürecinin diğer modellere göre daha hassas olduğu gözlemlenmiştir. Daha sonra, model üzerinde fine tuning uygulanmış; özellikle son inception blokları ve sınıflandırma katmanı eğitime açılmıştır.

Hiperparametre Optimizasyonu ve Deneyler

Hiperparametre optimizasyonu kapsamında, öğrenme oranı olarak 0.001 ve 0.0005 değerleri denenmiş, batch size 64 olarak seçilmiş, optimizör olarak Adam kullanılmıştır. Yapılan denemeler sonucunda öğrenme oranı 0.0005, batch size 64 ve Adam optimizör içeren konfigürasyon en stabil sonuçları vermiştir.

```
Final Config: Batch = 64 | LR = 0.0005 | Opt = adam
InceptionV3 final training hazır (freeze=True)
```

Bu yapılandırma ile InceptionV3, test seti üzerinde yaklaşık %76,12 doğruluk, %79,39 macro precision, %77,46 macro recall ve %77,49 macro F1-score değerlerine ulaşmıştır.

```
===== InceptionV3 TEST SONUÇLARI =====
Accuracy : 0.7612
Precision : 0.7939
Recall    : 0.7746
F1-score  : 0.7749
```

Modellerin Genel Karşılaştırması ve Performans Analizi

Bu çalışmada hem vize aşamasında temel CNN tabanlı modeller hem de final aşamasında ileri derin öğrenme mimarileri uygulanarak performans karşılaştırması yapılmıştır. Amaç, farklı mimari yapıların Türk İşaret Dili veri seti üzerindeki genelleme gücü, sınıf ayırt etme kabiliyeti ve eğitim sonrası doğruluk seviyelerini değerlendirmek ve en iyi modeli belirlemektir.

Modeller, aynı veri işleme boru hattı kullanılarak adil şartlarda test edilmiştir. Test metrikleri olarak Accuracy, Precision, Recall ve F1-Score (macro) kullanılmıştır.

GENEL KARŞILAŞTIRMA TABLOSU					
	Model	Accuracy	Precision	Recall	F1
0	DenseNet121	0.9988	0.9989	0.9989	0.9989
1	EfficientNetB0	0.9697	0.9716	0.9714	0.9713
2	MobileNetV2	0.9988	0.9989	0.9989	0.9989
3	ResNet18	0.9967	0.9969	0.9969	0.9969
4	ResNet50	0.9955	0.9952	0.9952	0.9952
5	ShuffleNetV2	0.9981	0.9982	0.9982	0.9982
6	VGG16	0.9939	0.9943	0.9942	0.9942
7	InceptionV3	0.7612	0.7939	0.7746	0.7749
8	AlexNet	0.9327	0.9354	0.9327	0.9330
9	LeNet5	0.8727	0.8223	0.8922	0.8794
10	CustomCNN1	0.9532	0.9359	0.9487	0.9499
11	CustomCNN2	0.8232	0.8123	0.8245	0.8145

-- Tabloda görüldüğü gibi MobileNetV2 ve DenseNet121 en yüksek başarıyı elde etmiş ve %99,89 seviyesinde macro-F1 skoruna ulaşmıştır.

--ResNet18 ve ResNet50 yakın sonuçlar vermiş, ancak parametre bakımından daha hafif olan ResNet18 oldukça verimli görünmektedir.

--EfficientNetB0 yüksek doğruluk üretmiş ancak %97 seviyesinde kalmıştır. Ayrıca InceptionV3, veri dağılımına bağlı olarak %76 seviyelerinde diğer modellere göre düşük bir performans sergilemiştir.

--Bu sonuçlar, ileri seviye mimarilerin özellik haritalama kabiliyetinin ve derinlik avantajının büyük bir fark yarattığını göstermektedir.

Sınıf Bazlı Model Performans Karşılaştırması

Her sınıf için modellerin F1 skorlarının yer aldığı tablo aşağıda sunulmuştur. Bu tablo, hangi modelin hangi harfi/niteliği daha iyi ayırt ettiğini ortaya koymaktadır.

	Class	DenseNet121	EfficientNetB0	MobileNetV2	ResNet18	ResNet50	ShuffleNetV2	VGG16	InceptionV3	AlexNet	LeNet5	CustomCNN1	CustomCNN2
0	A	99.795082	99.795082	99.795082	99.846154	99.539877	99.795082	99.743458	70.345500	94.000000	85.000000	94.000000	79.000000
1	B	99.974073	99.974073	99.974073	99.948160	99.974073	99.974073	99.321503	92.000000	97.000000	92.000000	98.000000	81.000000
2	C	99.923215	99.923215	99.923215	99.795082	99.846311	99.923215	99.769880	71.000000	95.000000	90.000000	96.000000	82.000000
3	D	99.845917	99.845917	99.845917	98.106539	99.743326	99.845917	98.542426	63.000000	93.000000	81.000000	93.000000	73.000000
4	E	99.948560	99.948560	99.948560	99.974273	99.897172	99.948560	99.614693	77.000000	93.000000	89.000000	96.000000	84.000000
5	F	99.949341	99.949341	99.949341	99.873257	99.898734	99.949341	99.696203	81.000000	94.000000	89.000000	96.000000	83.000000
6	G	99.948613	99.948613	99.948613	99.845996	99.845758	99.948613	98.778626	71.000000	93.000000	86.000000	94.000000	75.000000
7	H	99.923136	99.923136	99.923136	99.974379	99.614693	99.923136	99.820559	69.000000	95.000000	88.000000	94.000000	77.000000
8	I	99.922420	99.922420	99.922420	99.741736	99.844640	99.922420	99.612703	75.000000	93.000000	89.000000	96.000000	77.000000
9	J	99.897066	99.897066	99.897066	99.640103	99.536560	99.897066	99.146625	65.000000	88.000000	84.000000	95.000000	74.000000
10	K	100.000000	100.000000	100.000000	100.000000	99.897331	100.000000	99.514935	81.000000	95.000000	90.000000	96.000000	84.000000
11	L	99.897907	99.897907	99.897907	99.642492	99.795918	99.897907	99.338422	69.000000	92.000000	81.000000	94.000000	70.000000
12	M	99.355172	99.355172	99.355172	99.201648	98.183679	99.355172	98.011219	67.000000	89.000000	77.000000	84.000000	74.000000
13	N	99.302506	99.302506	99.302506	99.172271	98.022893	99.302506	97.856769	70.000000	88.000000	79.000000	87.000000	64.000000
14	O	99.974300	99.974300	99.974300	99.948613	99.922899	99.974300	99.251226	74.000000	94.000000	86.000000	96.000000	80.000000
15	P	99.872155	99.872155	99.872155	99.923254	99.923215	99.872155	99.821018	84.000000	96.000000	91.000000	96.000000	88.000000
16	R	99.793388	99.793388	99.793388	99.741869	99.767382	99.793388	99.559928	70.000000	95.000000	86.000000	96.000000	81.000000
17	S	99.948849	99.948849	99.948849	99.743852	98.836307	99.948849	99.288618	80.000000	93.000000	88.000000	94.000000	79.000000
18	T	99.948901	99.948901	99.948901	98.261997	98.614261	99.948901	99.564884	69.000000	91.000000	87.000000	96.000000	74.000000
19	U	99.974033	99.974033	99.974033	99.896212	99.974033	99.974033	99.688474	80.000000	96.000000	91.000000	98.000000	85.000000
20	V	99.974313	99.974313	99.974313	99.716714	99.922939	99.974313	99.845838	76.000000	90.000000	83.000000	95.000000	77.000000
21	Y	100.000000	100.000000	100.000000	99.948797	99.948797	100.000000	99.846469	81.000000	94.000000	86.000000	97.000000	85.000000
22	Z	99.948875	99.948875	99.948875	99.948901	99.948875	99.948875	99.390554	77.000000	89.000000	90.000000	96.000000	82.000000
23	del	100.000000	100.000000	100.000000	100.000000	98.425597	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000
24	nothing	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000
25	space	100.000000	100.000000	100.000000	100.000000	98.473658	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000

-- Custom modeller harf sınıflarında belirli yerlerde güçlü olsa da genel tutarlılık düşük kalmıştır.

-- DenseNet121, EfficientNet ve MobileNetV2 çoğu sınıfta 1.00 F1 Score elde ederek mükemmel yakın sınıflandırma yapmıştır. ResNet ailesi liderliği zaman zaman yakalasa da en üst üç model kadar baskın değildir.

-- InceptionV3 özellikle M, H, D, T gibi karmaşık görsel yapıyı işaretlerde daha düşük sonuçlar üretmiştir.

Hangi Model Hangi Sınıfta Lider?

Bu çalışma için oluşturulan analizde her sınıfta en yüksek F1 skoruna ulaşan model işaretlenmiştir.

	Class	Best Model(s)	Best F1
0	A	ResNet18	99.85
1	B	DenseNet121, EfficientNetB0, MobileNetV2, ResN...	99.97
2	C	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.92
3	D	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.85
4	E	ResNet18	99.97
5	F	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.95
6	G	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.95
7	H	ResNet18	99.97
8	I	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.92
9	J	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.90
10	K	DenseNet121, EfficientNetB0, MobileNetV2, ResN...	100.00
11	L	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.90
12	M	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.36
13	N	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.30
14	O	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.97
15	P	ResNet18	99.92
16	R	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.79
17	S	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.95
18	T	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.95
19	U	DenseNet121, EfficientNetB0, MobileNetV2, ResN...	99.97
20	V	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	99.97
21	Y	DenseNet121, EfficientNetB0, MobileNetV2, Shuf...	100.00
22	Z	ResNet18	99.95
23	del	DenseNet121, EfficientNetB0, MobileNetV2, ResN...	100.00
24	nothing	DenseNet121, EfficientNetB0, MobileNetV2, ResN...	100.00
25	space	DenseNet121, EfficientNetB0, MobileNetV2, ResN...	100.00

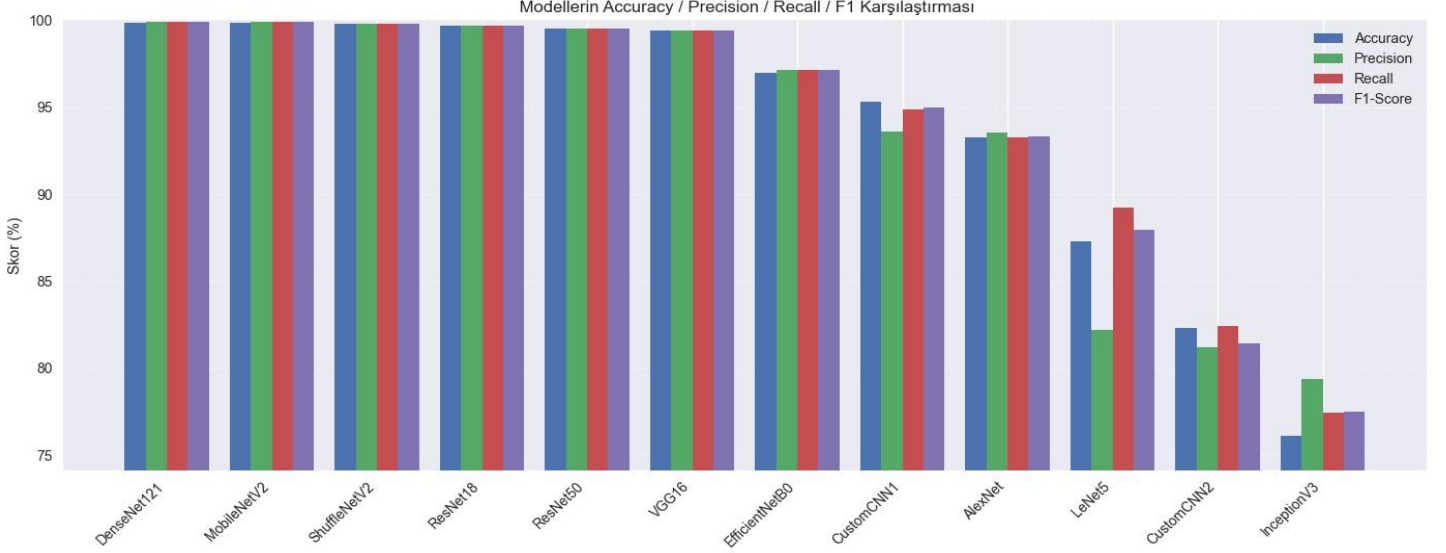
	Model	Leader_Class_Count
0	DenseNet121	21
1	EfficientNetB0	21
2	MobileNetV2	21
3	ShuffleNetV2	21
4	ResNet18	9
5	ResNet50	3
6	VGG16	3
7	InceptionV3	3
8	AlexNet	3
9	LeNet5	3
10	CustomCNN1	3
11	CustomCNN2	3

Bu çalışma için oluşturulan analizde her sınıfta en yüksek F1 skoruna ulaşan model işaretlenmiştir. DenseNet121, MobileNetV2, ShuffleNetV2 çoğu sınıfta liderdir ve birlikte mükemmel eşleşme göstermektedir. ResNet ailesi liderliği zaman zaman yakalasa da en üst üç model kadar baskın değildir. Basit CNN modelleri ve Inception, yalnızca bazı sınıflarda rekabet edebilmiştir. Bu bulgu, derin bağlantı yapısına sahip (DenseNet), mobil optimize edilmiş (MobileNet) ve hafif yapıda olmasına rağmen etkili (ShuffleNet) modellerin bu veri setine son derece uygun olduğunu göstermektedir.

Sonuçlar ve Tartışma

Bu çalışmada 26 sınıftan oluşan el işareti veri seti üzerinde farklı mimariler karşılaştırılmış ve modeller hem doğruluk (Accuracy), hem de sınıf bazlı metriklerle değerlendirilmiştir. Eğitim sürecinde ilk aşamada klasik modeller (AlexNet, LeNet5, CustomCNN1-2), final aşamasında ise gelişmiş derin mimariler (VGG16, ResNet18/50, DenseNet121, MobileNetV2, ShuffleNetV2, EfficientNetB0, InceptionV3) uygulanmıştır.

Aşağıdaki iki grafik, modellerin genel performans profilini göstermektedir:



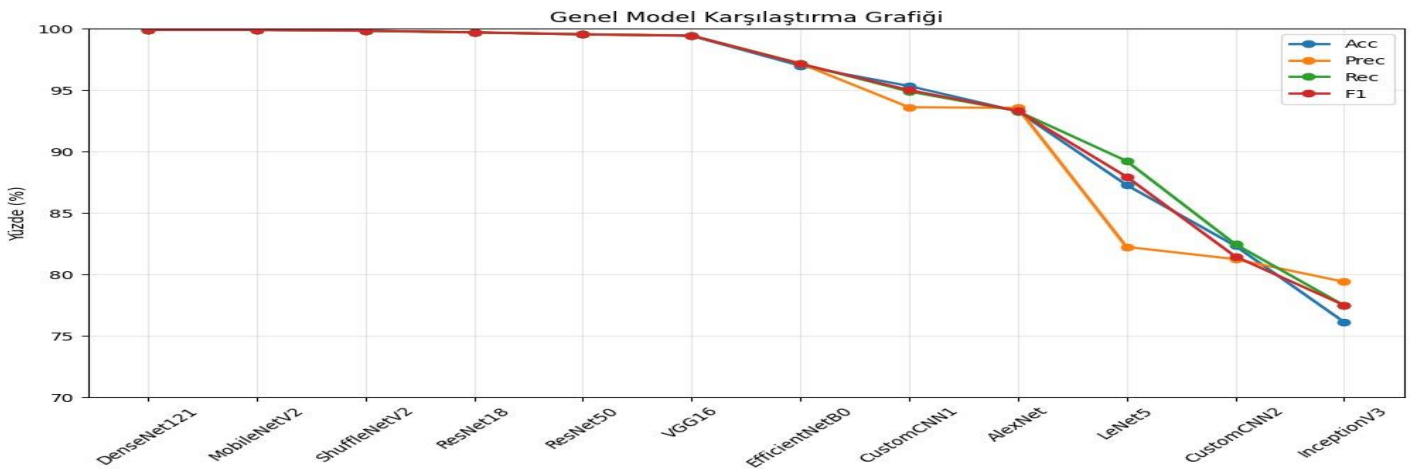
Grafik incelendiğinde modeller arasında belirgin bir performans ayrışması görülmektedir. Fine-tuning uygulanan transfer learning tabanlı modeller (DenseNet121, MobileNetV2, ShuffleNetV2 vb.) başarı oranında açık şekilde öne çıkarken; erken dönem vize modelleri olan AlexNet, LeNet5 ve Custom CNN mimarileri daha düşük başarı göstermiştir.

En yüksek performansı sergileyen modeller

DenseNet121 = %99.88

MobileNetV2 = %99.88

ShuffleNetV2 = %99.82



Bu grafikte model performans değişimini daha net göstermektedir. DenseNet/MobileNet/ShuffleNet çizgileri grafiğin tepesine stabil şekilde otururken, AlexNet → LeNet5 → Custom CNN → InceptionV3 yönünde belirgin bir düşüş görülmektedir. Bu düşüş özellikle F1 Score değerlerinde daha net fark yaratmaktadır. Lightweight modeller (MobileNetV2, ShuffleNetV2) hem hızlı hem yüksek doğruluk sunmuştur

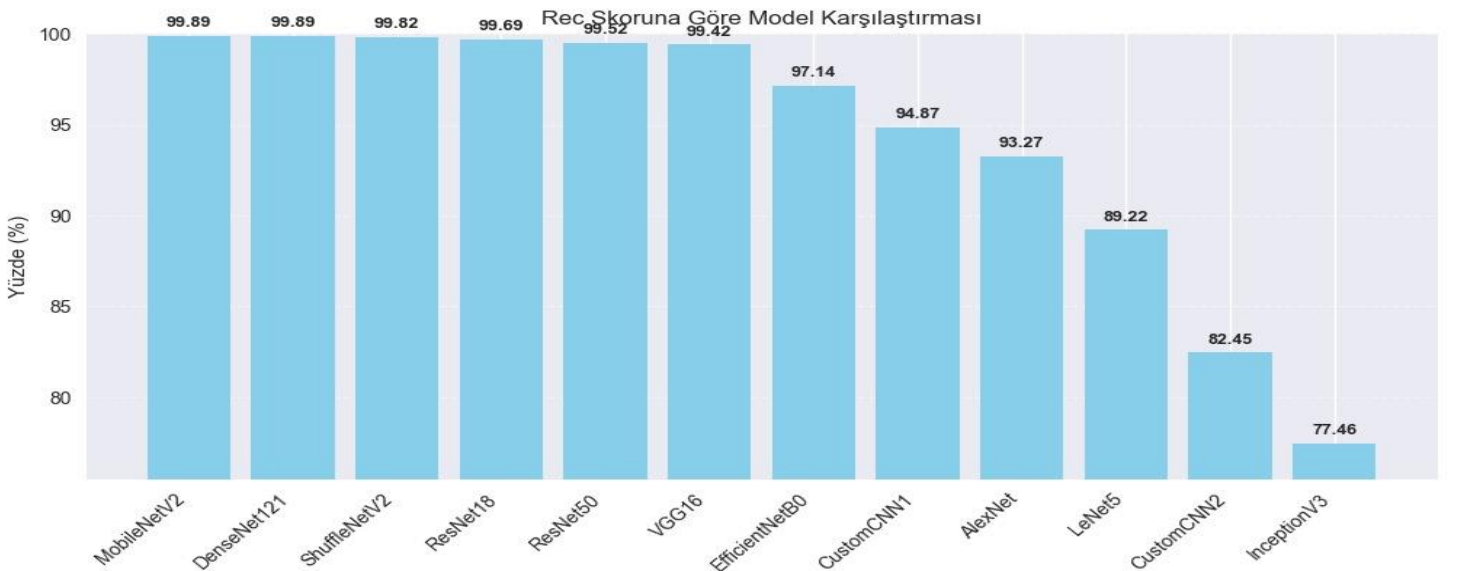
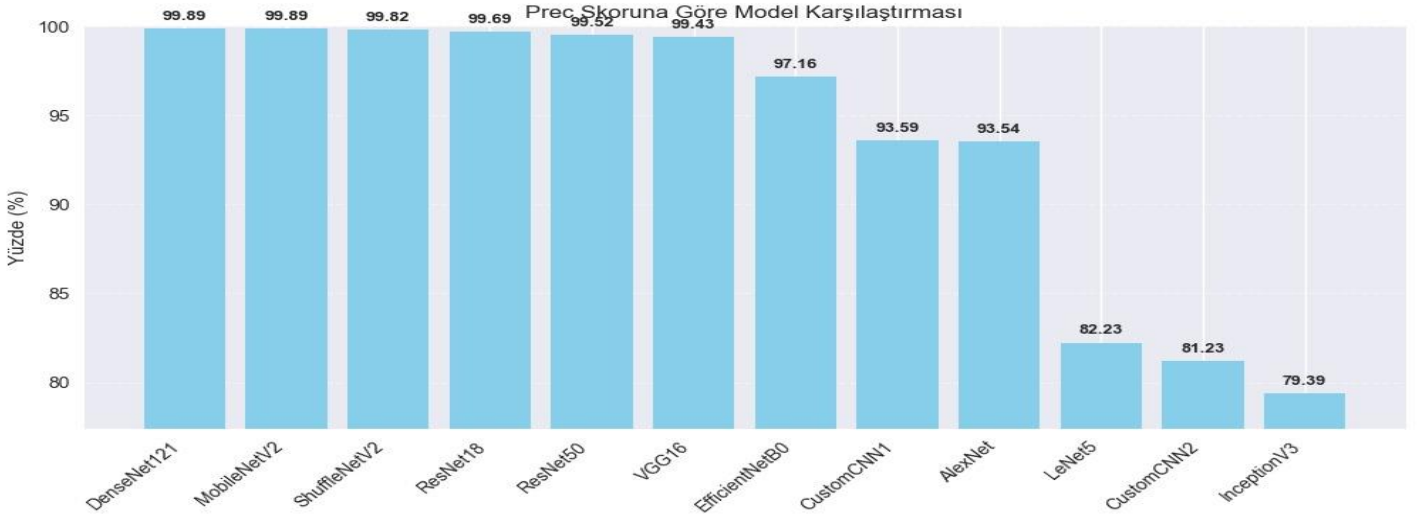
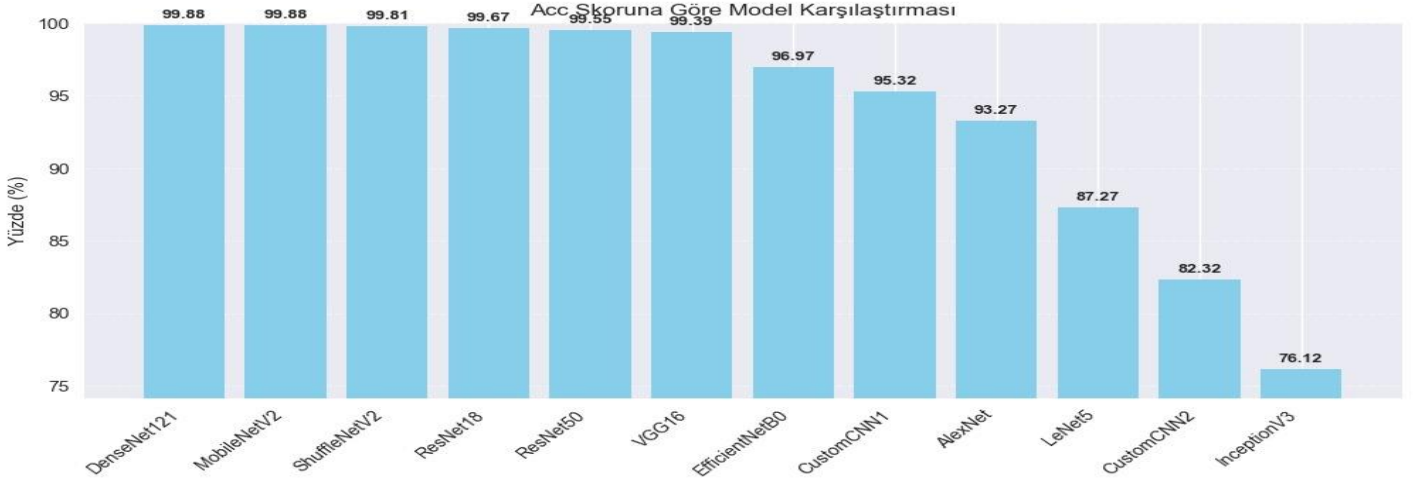
Kullanılan Araçlar ve Frameworkler

```
--- KULLANILAN KÜTÜPHANE VERSİYONLARI ---
1. Python Versiyonu: 3.10.1
-----
2. NumPy Versiyonu: 2.2.6
3. Pandas Versiyonu: 2.3.3
4. Scikit-learn Vers.: 1.7.2
5. PyTorch Versiyonu: 2.7.1+cu118
6. Torchvision Vers.: 0.22.1+cu118
```

Kullanılan seed:42

Github Repo Bağlantısı: <https://github.com/Byte52/DeepSign-Proje>

Ekler (Appendix)



KAYNAKLAR

- Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.
- A. Krizhevsky, I. Sutskever and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," in *Advances in Neural Information Processing Systems 25 (NIPS)*, 2012, pp. 1097-1105.
- G. Huang, Z. Liu, L. van der Maaten and K. Q. Weinberger, "Densely Connected Convolutional Networks," CVPR, 2017. (*DenseNet*)
- K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, 2016. (*ResNet*)
- A. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017.
- N. Ma, X. Zhang, H. Zheng and J. Sun, "ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design," ECCV, 2018.
- M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML, 2019.
- C. Szegedy et al., "Rethinking the Inception Architecture for Computer Vision," CVPR, 2016. (*InceptionV3*)
- Kaggle, "TR Sign Language Dataset," *Kaggle*, 2023. [Online]. Available: <https://www.kaggle.com/datasets/berkaykocaoglu/tr-sign-language>